

NxtGen Traders — AI Implementation Plan (v2.0)

Feed this document to an AI coding assistant (Claude, Cursor, Copilot, etc.) to scaffold and build the full application, phase by phase. **v2.0 adds explicit guardrails, a Definition-of-Done for every phase, and a data-consistency contract — these exist specifically to stop an AI agent from shipping half-finished screens, mismatched numbers, or silently-broken states.**

0. MASTER CONTEXT (Read Before Every Phase)

You are building **NxtGen Traders** — a Progressive Web App (PWA) for Indian stock market **paper trading** (simulated, no real money). Users start with a virtual ₹1,00,000 balance. The app targets beginner-to-advanced traders wanting a risk-free practice environment.

Non-negotiables

- All trades are virtual/paper trades. **No real money, no real broker order routing, ever.**
- Data is India-focused: NSE & BSE symbols, INR (₹) currency, IST (UTC+5:30) timestamps everywhere.
- Installable PWA on Android and iOS (Safari "Add to Home Screen").
- Dark-first, professional trading-terminal aesthetic — not a generic consumer app.
- Every screen must display a **SEBI-style disclaimer** somewhere in the footer/onboarding: *"NxtGen Traders is a paper trading simulator for educational purposes only. No real funds are involved. This is not investment advice."* This is a hard requirement, not optional polish — Indian fintech-adjacent apps should not appear to solicit real trading without this.

Non-goals (explicitly out of scope — do not build these even if it seems natural)

- Real brokerage integration, KYC/PAN verification, or real fund transfers.
 - Real order routing to NSE/BSE.
 - Anything that could be mistaken for a real trading terminal without the disclaimer visible.
-

1. AI AGENT GUARDRAILS (New — Read This Before Writing Any Code)

These rules exist because incomplete AI-generated builds tend to fail in the *same*

predictable ways: blank panels, mismatched numbers between screens, and inconsistent labels. Follow them on every phase, not just once.

- 1. **Never ship an empty gray box.** Every data-driven container (chart, list, panel) must be in exactly one of three states at all times: `(loading)` (skeleton, not a spinner-only or blank div), `(empty)` (icon + message + CTA), or `(populated)`. A container with no visible content and no state indicator is a bug, full stop.
- 2. **Single source of truth for every derived number.** Trade count, win rate, P&L, and portfolio value must each be computed by exactly one hook/function and reused everywhere they appear (Dashboard, Portfolio, Profile, Leaderboard). Do not let two screens independently recompute the same metric — this is how numbers drift out of sync between pages.
- 3. **Never fabricate placeholder data silently.** If live market data is unavailable, show a clearly labeled "Data delayed" or "Last known price" state — never render a fabricated number as if it were live, and never render `(0.00)` for a value that is actually "unknown"/"market closed" (those are different states and must look different).
- 4. **Consistent CTA copy.** The same user action must use the same button label everywhere (e.g., always "Explore Stocks," never "Explore" on one screen and "Explore Stocks" on another). Maintain a single copy/constants file for repeated CTA strings.
- 5. **No silent scope-cutting.** If a phase's requirements can't be fully completed, say so explicitly in code comments and in your final summary — do not quietly omit a required section (e.g., skipping the order-entry panel while completing the chart).
- 6. **Self-check against the phase's Definition of Done before declaring a phase complete.** Every phase in Section 8 ends with an explicit checklist — verify each line, don't assume.
- 7. **Ask before guessing on ambiguity.** If a requirement is genuinely ambiguous (e.g., which API to prioritize for a specific data point), state your assumption explicitly in a comment rather than guessing silently and moving on.
- 8. **Stay inside the locked stack (Section 2).** Do not introduce a new charting library, state manager, or CSS framework because it seems convenient for one feature.

2. TECH STACK (Locked — Do Not Change)

Layer	Choice
Frontend Framework	React 18 + TypeScript (strict mode, <code>(noImplicitAny: true)</code>)
Build Tool	Vite 5+

Layer	Choice
Styling	Tailwind CSS 3+ (JIT)
State Management	Zustand (UI/global state only) + React Query (all server state)
Charts	Lightweight Charts by TradingView (free, open-source)
Backend / Auth / DB	Supabase (PostgreSQL + Realtime + Auth + Edge Functions)
PWA	vite-plugin-pwa + Workbox
Icons	Lucide React
Notifications	Web Push API via Supabase Edge Function
Payments	Razorpay (Premium subscription)
Testing	Vitest + React Testing Library
Linting	ESLint + Prettier
Deployment	Vercel (Frontend) + Supabase Cloud

Rule: React Query owns *anything that comes from the network or database*. Zustand owns *anything that is purely client-side UI state* (theme, modal open/closed, active tab). Mixing these (e.g., caching server data in Zustand) is a common AI-agent mistake — do not do it.

3. DESIGN TOKENS

These are placeholder/default tokens. If a final brand palette is supplied separately, replace the values below **without changing the token names** — every component should reference tokens, never hardcoded hex values, so a full re-theme never requires touching component code.

ts

```
// src/styles/tokens.ts

export const colors = {
  // Backgrounds (layered depth – use exactly these three levels, no ad-hoc g
  bgPrimary: '#0B0E14',      // App background
  bgSurface: '#141822',      // Card / panel background
  bgElevated: '#1C212E',     // Hover state, modals, dropdowns
  border: '#252B3B',         // Dividers, card borders

  // Brand
  brandPrimary: '#5B6EF5',    // Primary actions, active nav/tab, links
  brandPrimaryHover: '#7A8AFF',
  brandAccent: '#F0A93D',     // Reserved ONLY for premium/achievement/trophy

  // Semantic – SINGLE canonical set. Do not also define a separate "brandGre
  // gains/losses/status always resolve to exactly these four values.
  positive: '#00C896',        // Gains, BUY, success states
  positiveMuted: 'rgba(0,200,150,0.12)',
  negative: '#FF4D6A',        // Losses, SELL, error states
  negativeMuted: 'rgba(255,77,106,0.12)',
  neutral: '#8B93A7',         // Flat / 0.00% / unknown-but-not-an-error state
  warning: '#F0A93D',

  // Text
  textPrimary: '#F5F6FA',
  textSecondary: '#9AA3B8',
  textMuted: '#5C6478',
};

export const fonts = {
  display: '"Inter", "Roboto", sans-serif',      // Headings
  body: '"system-ui", "Segoe UI", sans-serif',   // Body text
  mono: '"JetBrains Mono", "Fira Code", monospace', // ALL numbers: prices, P
};
```

Rule: `positive`/`negative`/`neutral` are the *only* colors ever used for price/P&L direction. `neutral` (● `#8B93A7`) must be used — not green — whenever a value is genuinely flat, closed-market, or not-yet-loaded, so "flat" is never visually confused with "broken."

Tailwind config must extend `theme.colors` and `theme.fontFamily` with these tokens — never allow raw hex values inside component `className` strings.

4. PROJECT FOLDER STRUCTURE

```
nxtgen-traders/
├─ public/
│   └─ icons/                # PWA icons (72, 96, 128, 144, 152, 192,
384, 512px)
│   └─ manifest.json
│   └─ screenshots/
├─ src/
│   └─ api/
│       └─ marketData.ts    # NSE/BSE price data (with fallback chain -
see Section 10)
│       └─ news.ts
│       └─ supabase.ts
│   └─ components/
│       └─ ui/              # Button, Input, Card, Modal, Badge, Toast,
Spinner, EmptyState, Skeleton
│           └─ charts/      # CandleChart, LineChart, VolumeBar
│           └─ trading/     # OrderForm, OrderBook, TradeConfirmModal
│           └─ portfolio/   # HoldingsTable, PnLCard, AllocationPie
│           └─ watchlist/   # WatchlistCard, AlertModal
│           └─ layout/      # AppShell, BottomNav, TopBar, Sidebar
│           └─ common/      # LoadingSpinner, EmptyState,
ErrorBoundary, SEOHead
│   └─ hooks/
│       └─ useMarketData.ts
│       └─ usePortfolio.ts  # SINGLE source of truth for portfolio-
derived numbers
│       └─ useTradingStats.ts # SINGLE source of truth for trade count
/ win rate / best day
│       └─ useOrders.ts
│       └─ useWatchlist.ts
│       └─ useAuth.ts
│       └─ useRealtime.ts
│   └─ pages/              # Landing, Login, Register, Dashboard,
Charts, Portfolio, Orders,
│       └─                # Watchlist, Trade, News, Leaderboard,
Profile, Settings, Premium, Help
│   └─ store/              # authStore, uiStore, marketStore (UI-
state only, see Section 2 rule)
│   └─ styles/             # tokens.ts, globals.css
│   └─ types/              # market.ts, trade.ts, user.ts, api.ts
│   └─ utils/
│       └─ formatters.ts   # formatINR, formatPct, formatVolume,
```

```
formatDate (IST)
| | | └─ validators.ts
| | | └─ calculations.ts      # P&L, avg cost, win rate – imported by
useTradingStats ONLY
| | | └─ constants.ts        # Market hours, exchange codes, shared CTA
copy strings
| | └─ App.tsx / main.tsx / vite-env.d.ts
└─ supabase/
  | └─ migrations/
  | └─ functions/            # market-data, place-order, send-
notification, leaderboard, check-alerts
└─ index.html / vite.config.ts / tailwind.config.ts / tsconfig.json /
package.json
```

Rule: `calculations.ts` contains the math; `useTradingStats.ts` and `usePortfolio.ts` are the *only* hooks allowed to call it and expose the results. Any page needing trade count, win rate, or P&L imports one of these two hooks — never recomputes independently. This directly prevents the "Leaderboard shows 12 trades, Profile shows 42 trades" class of bug.

5. SUPABASE DATABASE SCHEMA

```
sql
```

```

-- =====
-- MIGRATION 001: Core user tables
-- =====

CREATE TABLE public.profiles (
  id UUID PRIMARY KEY REFERENCES auth.users(id) ON DELETE CASCADE,
  full_name TEXT NOT NULL,
  phone TEXT,
  avatar_url TEXT,
  is_premium BOOLEAN DEFAULT FALSE,
  premium_until TIMESTAMPTZ,
  created_at TIMESTAMPTZ DEFAULT NOW(),
  updated_at TIMESTAMPTZ DEFAULT NOW()
);

CREATE TABLE public.accounts (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id UUID NOT NULL REFERENCES public.profiles(id) ON DELETE CASCADE,
  balance NUMERIC(15,2) DEFAULT 100000.00,
  invested_value NUMERIC(15,2) DEFAULT 0.00,
  total_pnl NUMERIC(15,2) DEFAULT 0.00,
  created_at TIMESTAMPTZ DEFAULT NOW(),
  updated_at TIMESTAMPTZ DEFAULT NOW(),
  UNIQUE(user_id)
);

-- =====
-- MIGRATION 002: Market & trading tables
-- =====

CREATE TABLE public.holdings (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id UUID NOT NULL REFERENCES public.profiles(id) ON DELETE CASCADE,
  symbol TEXT NOT NULL,
  exchange TEXT NOT NULL CHECK (exchange IN ('NSE','BSE')),
  company_name TEXT NOT NULL,
  quantity INTEGER NOT NULL CHECK (quantity > 0),
  avg_buy_price NUMERIC(10,2) NOT NULL,
  created_at TIMESTAMPTZ DEFAULT NOW(),
  updated_at TIMESTAMPTZ DEFAULT NOW(),
  UNIQUE(user_id, symbol, exchange)
);

CREATE TABLE public.orders (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id UUID NOT NULL REFERENCES public.profiles(id) ON DELETE CASCADE,

```

```

symbol TEXT NOT NULL,
exchange TEXT NOT NULL CHECK (exchange IN ('NSE','BSE')),
company_name TEXT NOT NULL,
order_type TEXT NOT NULL CHECK (order_type IN ('MARKET','LIMIT','SL','SL-M'
side TEXT NOT NULL CHECK (side IN ('BUY','SELL')),
quantity INTEGER NOT NULL CHECK (quantity > 0),
price NUMERIC(10,2),
trigger_price NUMERIC(10,2),
status TEXT NOT NULL DEFAULT 'PENDING' CHECK (status IN ('PENDING','EXECUTE
executed_price NUMERIC(10,2),
executed_at TIMESTAMPTZ,
created_at TIMESTAMPTZ DEFAULT NOW()
);
CREATE INDEX idx_orders_user_status ON public.orders(user_id, status);

CREATE TABLE public.trades (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id UUID NOT NULL REFERENCES public.profiles(id),
  order_id UUID NOT NULL REFERENCES public.orders(id),
  symbol TEXT NOT NULL,
  exchange TEXT NOT NULL,
  side TEXT NOT NULL CHECK (side IN ('BUY','SELL')),
  quantity INTEGER NOT NULL,
  price NUMERIC(10,2) NOT NULL,
  pnl NUMERIC(10,2),          -- populated on SELL only; NULL on BUY (not zer
  traded_at TIMESTAMPTZ DEFAULT NOW()
);
CREATE INDEX idx_trades_user ON public.trades(user_id, traded_at DESC);

-- =====
-- MIGRATION 003: Watchlists
-- =====

CREATE TABLE public.watchlists (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id UUID NOT NULL REFERENCES public.profiles(id) ON DELETE CASCADE,
  name TEXT NOT NULL DEFAULT 'My Watchlist',
  created_at TIMESTAMPTZ DEFAULT NOW()
);

CREATE TABLE public.watchlist_items (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  watchlist_id UUID NOT NULL REFERENCES public.watchlists(id) ON DELETE CASCA
  symbol TEXT NOT NULL,
  exchange TEXT NOT NULL,
  company_name TEXT NOT NULL,

```



```

    added_at TIMESTAMPTZ DEFAULT NOW(),
    UNIQUE(watchlist_id, symbol, exchange)
);

CREATE TABLE public.alerts (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    user_id UUID NOT NULL REFERENCES public.profiles(id) ON DELETE CASCADE,
    symbol TEXT NOT NULL,
    exchange TEXT NOT NULL,
    condition TEXT NOT NULL CHECK (condition IN ('ABOVE', 'BELOW')),
    target_price NUMERIC(10,2) NOT NULL,
    is_active BOOLEAN DEFAULT TRUE,
    triggered_at TIMESTAMPTZ,
    created_at TIMESTAMPTZ DEFAULT NOW()
);

-- =====
-- MIGRATION 004: Leaderboard & notifications
-- =====

CREATE TABLE public.leaderboard_snapshots (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    user_id UUID NOT NULL REFERENCES public.profiles(id),
    full_name TEXT NOT NULL,
    avatar_url TEXT,
    total_return NUMERIC(8,4),
    total_pnl NUMERIC(15,2),
    win_rate NUMERIC(5,2),
    total_trades INTEGER,
    period TEXT CHECK (period IN ('DAILY', 'WEEKLY', 'MONTHLY', 'ALL_TIME')),
    rank INTEGER,
    snapshot_date DATE DEFAULT CURRENT_DATE,
    UNIQUE(user_id, period, snapshot_date)
);

-- IMPORTANT: total_trades and win_rate here MUST be computed with the exact
-- SQL/logic that useTradingStats.ts uses client-side for the Profile page, s
-- two screens can never disagree. Extract the calculation into a shared SQL
-- (see below) rather than duplicating the formula in the Edge Function.

CREATE VIEW public.user_trading_stats AS
SELECT
    user_id,
    COUNT(*) AS total_trades,
    ROUND(100.0 * COUNT(*) FILTER (WHERE pnl > 0) / NULLIF(COUNT(*) FILTER (WHE
    MAX(pnl) AS best_trade_pnl
FROM public.trades

```

```

GROUP BY user_id;
-- Both the Profile page and the Leaderboard Edge Function must read from this table
-- (or an identical query) – never hand-roll the count/win-rate math in two places

CREATE TABLE public.push_subscriptions (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id UUID NOT NULL REFERENCES public.profiles(id) ON DELETE CASCADE,
  endpoint TEXT NOT NULL,
  auth_key TEXT NOT NULL,
  p256dh_key TEXT NOT NULL,
  created_at TIMESTAMPTZ DEFAULT NOW(),
  UNIQUE(user_id, endpoint)
);

-- =====
-- ROW LEVEL SECURITY – apply to all tables
-- =====

ALTER TABLE public.profiles ENABLE ROW LEVEL SECURITY;
ALTER TABLE public.accounts ENABLE ROW LEVEL SECURITY;
ALTER TABLE public.holdings ENABLE ROW LEVEL SECURITY;
ALTER TABLE public.orders ENABLE ROW LEVEL SECURITY;
ALTER TABLE public.trades ENABLE ROW LEVEL SECURITY;
ALTER TABLE public.watchlists ENABLE ROW LEVEL SECURITY;
ALTER TABLE public.watchlist_items ENABLE ROW LEVEL SECURITY;
ALTER TABLE public.alerts ENABLE ROW LEVEL SECURITY;
ALTER TABLE public.push_subscriptions ENABLE ROW LEVEL SECURITY;

CREATE POLICY "own_profile" ON public.profiles FOR ALL USING (auth.uid() = id);
CREATE POLICY "own_account" ON public.accounts FOR ALL USING (auth.uid() = user_id);
CREATE POLICY "own_holdings" ON public.holdings FOR ALL USING (auth.uid() = user_id);
CREATE POLICY "own_orders" ON public.orders FOR ALL USING (auth.uid() = user_id);
CREATE POLICY "own_trades" ON public.trades FOR ALL USING (auth.uid() = user_id);
CREATE POLICY "own_watchlist" ON public.watchlists FOR ALL USING (auth.uid() = user_id);
CREATE POLICY "own_alerts" ON public.alerts FOR ALL USING (auth.uid() = user_id);
CREATE POLICY "public_leaderboard" ON public.leaderboard_snapshots FOR SELECT

```

6. CORE TYPE DEFINITIONS

ts

```

// src/types/market.ts
export interface Stock {
  symbol: string;
  exchange: 'NSE' | 'BSE';
  companyName: string;
  sector: string;
  isin: string;
}

export interface Quote {
  symbol: string;
  exchange: 'NSE' | 'BSE';
  ltp: number;
  open: number;
  high: number;
  low: number;
  close: number;
  change: number;
  changePct: number;
  volume: number;
  timestamp: string; // ISO string, always stored/converted via IST utility
  isStale: boolean; // true if this quote is a cached/fallback value,
}

export interface OHLCV {
  time: number;
  open: number; high: number; low: number; close: number; volume: number;
}

export type Timeframe = '1m' | '5m' | '15m' | '30m' | '1h' | '1d' | '1w' | '1m'

// src/types/trade.ts
export type OrderType = 'MARKET' | 'LIMIT' | 'SL' | 'SL-M';
export type OrderSide = 'BUY' | 'SELL';
export type OrderStatus = 'PENDING' | 'EXECUTED' | 'CANCELLED' | 'REJECTED';

export interface Order {
  id: string; userId: string; symbol: string; exchange: 'NSE' | 'BSE'; comparison: string;
  orderType: OrderType; side: OrderSide; quantity: number; price?: number; triggerPrice?: number;
  status: OrderStatus; executedPrice?: number; executedAt?: string; createdAt: string;
}

export interface Holding {
  id: string; symbol: string; exchange: 'NSE' | 'BSE'; companyName: string; currentPrice: number;

```

```
    avgBuyPrice: number; currentPrice?: number; currentValue?: number; invested
    pnl?: number; pnlPct?: number;
  }

// src/types/user.ts
export interface UserProfile {
  id: string; fullName: string; phone?: string; avatarUrl?: string;
  isPremium: boolean; premiumUntil?: string;
}

export interface Account {
  balance: number; investedValue: number; totalPnl: number; totalPortfolioVal
}

export interface TradingStats {
  totalTrades: number; winRate: number; bestTradePnl: number;
}
// TradingStats is the ONLY shape used to render trade counts / win rate anyw
```

Rule: No `(any)`. If a third-party API response shape is unknown, define an explicit `(unknown)`-based type and narrow it — do not silently type it as `(any)` to move faster.

7. ENVIRONMENT VARIABLES

```
bash
```

```

# Supabase
VITE_SUPABASE_URL=https://your-project.supabase.co
VITE_SUPABASE_ANON_KEY=your_anon_key

# Market Data – try in this order, see Section 10 for fallback strategy
VITE_TWELVE_DATA_KEY=your_key
VITE_ALPHA_VANTAGE_KEY=your_key
VITE_FINNHUB_KEY=your_key
VITE_USE MOCK_DATA=false           # set true in local dev if no API keys are co

# News
VITE_NEWS_API_KEY=your_key

# Payments
VITE_RAZORPAY_KEY_ID=rzp_test_xxxxxxx

# PWA Push
VITE_VAPID_PUBLIC_KEY=your_vapid_public_key

# App
VITE_APP_URL=http://localhost:5173
VITE_APP_NAME=NxtGen Traders

```

Never commit `.env`. Provide `.env.example` with empty values checked into git.

8. PHASE-BY-PHASE DEVELOPMENT PLAN

Every phase below ends with a **Definition of Done** — treat these as a literal checklist, not a vibe check.

PHASE 1 — Project Bootstrap & Foundation (Days 1-2)

Scaffold Vite + React + TS, Tailwind (with tokens from Section 3, `darkMode: 'class'`), full folder structure, React Router v6 with all routes, Supabase client singleton, `AppShell` with `BottomNav` (Home/Charts/Portfolio/Watchlist/More) + `TopBar`, theme toggle via `uiStore`, and these atomic components: `Button`, `Input`, `Card`, `Badge`, `Modal`, `Toast`, `Spinner`, `Skeleton`, `EmptyState`. Run migrations from Section 5. Configure PWA plugin.

Definition of Done:

- ☐ `npm run dev` starts with zero console errors

- ☐ All routes render (placeholder pages acceptable, but each placeholder uses `EmptyState`, not a blank div)
- ☐ Dark/light toggle works and persists across reload
- ☐ `/dev/components` page demonstrates every atomic component in all its variants, including `Skeleton` and `EmptyState`

PHASE 2 — Authentication (Days 3-4)

`useAuth` wrapping Supabase Auth (`signIn`, `signInWithPassword`, `signInWithOAuth('google')`, `signOut`, `resetPassword`). `authStore` populated on `onAuthStateChange`. Landing, Login, Register pages per original spec. `ProtectedRoute` / `PublicOnlyRoute`. On first login, create `accounts` row with ₹1,00,000 balance **only if one doesn't already exist** (idempotent — do not risk resetting balance on repeat logins).

Definition of Done:

- ☐ Register → email confirm → login → lands on `/dashboard`
- ☐ Protected routes redirect unauthenticated users; public-only routes redirect authenticated users
- ☐ Google OAuth completes end-to-end
- ☐ Session survives a hard page refresh
- ☐ Re-logging in an existing user does **not** reset their balance

PHASE 3 — Market Data Layer (Days 5-6)

`marketData.ts` with `getQuote`, `getQuotes`, `getCandles`, `searchStocks`, `getTopMovers`, `getIndices`. Fallback chain: Twelve Data → Alpha Vantage → cached last-known value → mock generator (see Section 10 for why this order and why mock data is a legitimate last resort, not a cop-out). React Query hooks with the specified refetch intervals. `isMarketOpen()` util using IST market hours (9:15-15:30, Mon-Fri, and skip known NSE holidays if a holiday list is available; otherwise weekday-only is an acceptable v1).

Definition of Done:

- ☐ `useQuote('RELIANCE', 'NSE')` returns a `Quote` with `isStale` correctly set to `false` for live data and `true` for fallback data
- ☐ Every quote/candle consumer checks `isStale` and renders a "delayed" indicator when true — this is not optional
- ☐ Closed-market state shows last close value with a "Market Closed" badge, never a bare `0.00`
- ☐ `formatINR` produces `₹1,23,456.78` (Indian digit grouping), verified with a unit test

PHASE 4 — Dashboard (Day 7)

Account summary bar, market indices strip, top gainers/losers, portfolio preview (uses `usePortfolio`), not its own calculation), recent orders, news preview, quick-action FAB. Skeletons for every section while loading; `EmptyState` for every section with no data — **the central chart/news panels must never render as an empty container** (this was the single biggest issue in the previous version of this app).

Definition of Done:

- ☐ Every one of the 6 dashboard sections is, at all times, in `loading` / `empty` / `populated` state — verified by manually testing with an empty account and a throttled network
- ☐ Portfolio preview numbers exactly match the numbers shown on `/portfolio` (same hook, same values)
- ☐ Indices strip never shows `0.00 (0.00%)` for a closed market — shows last close + "Market Closed" badge instead

PHASE 5 — Advanced Charting (Days 8-10)

Lightweight Charts integration, symbol search, timeframe pills, chart type toggle, indicators panel (SMA/EMA/RSI/MACD/Bollinger via the `technicalindicators` package or hand-rolled TS), drawing tools, stock info header, quick order button, URL-param persistence (`?symbol=&exchange=&tf=`).

Definition of Done:

- ☐ Chart loads with real (or clearly-labeled mock) data for any searched NSE symbol on page load — never opens to a blank canvas
- ☐ Timeframe switching re-fits the chart and preserves the selected symbol
- ☐ At minimum SMA, RSI, and Volume render correctly
- ☐ Quick-order button opens the order modal without losing chart state

PHASE 6 — Trading Engine (Days 11-13)

`trading.ts` (`placeOrder`, `cancelOrder`, `getOrders`, `getHoldings`, `getAccount`) with the market/limit execution logic exactly as specified in the original plan, run as an **atomic Supabase transaction** (all-or-nothing — balance deduction, order insert, trade insert, and holdings upsert must not partially apply on failure). `/trade/:symbol` screen with order form, confirmation modal, optimistic UI update reconciled against the server response. Validation: min qty 1, min order value ₹500, sufficient balance/holdings, limit price within $\pm 20\%$ of LTP.

Definition of Done:

- ☐ A market BUY instantly and correctly updates balance, holdings, and the recent-orders list — with no partial-failure state possible (test by simulating a mid-transaction error)
- ☐ A SELL correctly computes P&L and never allows selling more than currently held
- ☐ Optimistic UI updates roll back cleanly if the server rejects the order
- ☐ Every validation rule produces a specific, inline error message — never a silent failure

PHASE 7 — Portfolio Management (Day 14)

Summary cards, holdings table, allocation pie (Recharts), performance graph (7d/30d/90d), stats section (via `useTradingStats`, not a separate calculation), CSV export.

Definition of Done:

- ☐ Stats section numbers match Profile page numbers exactly (same underlying view/hook — see Section 5's `user_trading_stats` view)
- ☐ Holdings table P&L color-codes correctly and is keyboard-navigable
- ☐ Empty portfolio shows the standard `EmptyState`, not a blank table

PHASE 8 — Watchlist (Day 15)

Multi-list tabs, add/delete list (3 max on free tier), per-row live price + sparkline + quick buy/sell, alert modal, `check-alerts` Edge Function (1-min cron).

Definition of Done:

- ☐ Adding/removing a stock updates instantly (optimistic) and persists on refresh
- ☐ Alerts fire a push notification within ~1 minute of the condition being met (test manually with a tight threshold)

PHASE 9 — News Feed (Day 16)

Category pills, news cards, bookmarking (localStorage), infinite scroll, related-stock chips, 5-min auto-refresh.

Definition of Done:

- ☐ Dashboard's news preview and the full News page pull from the same `news.ts` function/cache — never two separate implementations
- ☐ Bookmark state persists across reload

PHASE 10 — Leaderboard (Day 17)

`update-leaderboard` Edge Function (daily cron) reading from `user_trading_stats` view. Period tabs, podium, ranked list, current-user row always visible (highlighted even if outside top 100), rank-change indicator, achievement badges.

Definition of Done:

- ☐ Leaderboard's `total_trades` / `win_rate` for the current user is byte-for-byte identical to what Profile shows for the same period — this is the exact bug class to eliminate; write a test asserting both read paths return the same numbers for a seeded user
- ☐ Rank-change indicator has a legible legend (tooltip or key) explaining $\uparrow$ / $\downarrow$ / —

PHASE 11 — Profile & Settings (Day 18)

Avatar upload, editable name/phone (masked by default, e.g. `7777•••162`), reveal-on-tap), stats summary (via `useTradingStats`), achievement badge grid with a visually distinct locked vs. unlocked state (grayscale + lock icon vs. full color), Settings page per original spec including theme, notifications, trading defaults, data export, reset portfolio, delete account.

Definition of Done:

- ☐ Locked and unlocked achievements are distinguishable at a glance without reading the label
- ☐ Phone number is masked by default
- ☐ "Reset portfolio" requires a confirmation modal and correctly restores ₹1,00,000 while clearing holdings/orders/trades for that user only

PHASE 12 — Premium Subscription (Day 19)

Feature comparison table, two plans, Razorpay checkout via `create-subscription` Edge Function, signature verification, `is_premium` flag update, premium feature gating via `authStore`.

Definition of Done:

- ☐ Test-mode Razorpay payment completes and `is_premium` flips to `true` within one page refresh
- ☐ Premium-gated features are inaccessible (not just visually hidden — actually blocked) for non-premium users

PHASE 13 — Help Center (Day 20)

FAQ accordion (15+ items), tutorials, searchable glossary, contact form, about/legal.

Definition of Done:

- ☐ Glossary search filters in real time with no layout shift

PHASE 14 — PWA, Offline & Performance (Day 21)

Full manifest, Workbox strategies (cache-first for static assets, network-first for API calls,

stale-while-revalidate for app shell/news), offline banner showing last-synced data, route-based code splitting, virtualized long lists, debounced search, custom "Add to Home Screen" prompt after 3rd session.

Definition of Done:

- ☐ Fully offline load shows cached portfolio/watchlist with a visible "Offline — showing last synced data" banner, never a blank/error screen
- ☐ Lighthouse PWA score > 90

PHASE 15 — Testing & Deployment (Day 22)

Unit tests for formatters/calculations/validators, integration tests for auth/trading/watchlist flows, GitHub Actions CI (lint + test on PR, build check on merge), Vercel deployment, Lighthouse audit gate (Performance > 90, PWA > 90, Accessibility > 85).

Definition of Done:

- ☐ CI blocks merge on lint or test failure
- ☐ All three Lighthouse thresholds pass on the deployed preview URL

9. DATA CONSISTENCY CONTRACT (New)

This section exists to eliminate the class of bug where the same metric shows different values on different screens.

Metric	Single Source	Consumers
Total trades	<code>user_trading_stats</code> view (SQL) via <code>useTradingStats</code> hook	Profile, Leaderboard
Win rate	<code>user_trading_stats</code> view via <code>useTradingStats</code>	Profile, Leaderboard
Portfolio current value / P&L	<code>usePortfolio</code> hook	Dashboard, Portfolio
Account balance	<code>useAuth</code> / <code>authStore.account</code> (refetched via React Query, not duplicated locally)	Dashboard, Trade screen, Order form

Rule: before adding any new screen that displays a number already computed elsewhere in this table, import the existing hook. Do not write a new `SELECT`/calculation for a

metric that already has a canonical source.

10. MARKET DATA RELIABILITY STRATEGY (New)

Real-time NSE/BSE coverage varies significantly across free-tier APIs — Alpha Vantage's India coverage is limited, and hitting any of these APIs directly from every client at a 5-second interval will exhaust free-tier rate limits almost immediately once you have more than a handful of concurrent users.

1. **Do not call market data APIs directly from the browser at scale.** Route quote/candle requests through a Supabase Edge Function (`market-data`) that fetches from the provider, caches the result for a short TTL (e.g., 3-5s during market hours, 60s otherwise), and serves all clients from that cache. This is a caching proxy, not optional infrastructure — it's what prevents the app from being rate-limited into showing broken data under real usage.
 2. **Fallback chain, in order:** Twelve Data → Alpha Vantage → last cached value in the DB/localStorage (marked `isStale: true`) → deterministic mock data generator (only in local dev, gated by `VITE_USE MOCK DATA`).
 3. **Mock data generator, when used, must be visually distinguishable** — e.g., a small "Simulated data" tag — so it's never confused with a live feed during a demo or by a real user.
 4. **Never let a market data failure crash a page.** Wrap every market-data-consuming component in the shared `ErrorBoundary`, and design each component to render its `EmptyState`/stale-data variant rather than throwing.
-

11. SUPABASE EDGE FUNCTIONS

ts

```
// 1. market-data/index.ts
// Caching proxy in front of Twelve Data / Alpha Vantage (see Section 10). All
// quote/candle requests go through this function, not directly to the provider.

// 2. place-order/index.ts
// Executes paper trades atomically: fetch live price, validate balance/holdings
// run all DB writes in a single transaction (all-or-nothing).

// 3. check-alerts/index.ts
// Cron: every 1 minute. Compares active alerts to live prices, fires push
// notifications, marks triggered alerts inactive.

// 4. update-leaderboard/index.ts
// Cron: daily at 6:00 PM IST. Reads from the `user_trading_stats` view (Section 10).
// does NOT recompute trade stats independently. Upserts snapshots per period.
// total_return DESC.

// 5. create-subscription/index.ts
// Creates Razorpay order, verifies webhook signature, sets profiles.is_premium
```

12. KEY IMPLEMENTATION NOTES FOR AI

1. All prices are virtual/paper trading only — no real broker API integration anywhere.
2. Indian number formatting is mandatory: `(en-IN)` locale, `(₹1,23,456.78)` not `(₹123,456.78)`.
3. IST throughout the UI; store timestamps in UTC in the DB and convert at render time via `(Intl.DateTimeFormat)` with `(timeZone: 'Asia/Kolkata')`.
4. Market-hours awareness drives both the UI indicator (green dot "Market Open" / grey "Market Closed") and polling frequency — these must stay in sync (don't poll every 5s while showing "Market Closed").
5. Graceful degradation over broken UI: if all market data sources fail, show the last known price with a visible "Data delayed" label — never a broken/blank component (see Sections 1 and 10).
6. Mobile-first: design for 390px width first, then tablet, then desktop.
7. Accessibility baseline: keyboard focus on all interactive elements, `(aria-label)` on icon-only buttons, color is never the sole indicator of state (always pair with text/icon — e.g., not just red/green but also a ▲/▼ glyph).
8. Monospace font for all financial numbers to prevent layout shift on live updates.
9. Optimistic UI for trade placement, reconciled against the server response; roll back cleanly on rejection.

10. Every page wrapped in `<ErrorBoundary>` with a friendly error card, never a crashed white screen.
 11. Every duplicated string (CTA labels, empty-state copy) lives in `constants.ts` — not re-typed per screen.
-

13. PROMPT TEMPLATE FOR EACH PHASE

You are building NxtGen Traders — an Indian stock market paper trading PWA.

Context:

- Tech stack: React 18 + TypeScript + Tailwind CSS + Vite + Supabase + Zustand + React Query
- Design tokens: [paste Section 3]
- Relevant types: [paste relevant types from Section 6]
- Folder structure: [paste Section 4]
- Supabase schema: [paste relevant tables/views from Section 5]
- Data consistency contract: [paste Section 9 if this phase displays any shared metric]

Now implement PHASE [N]: [Phase title]

Requirements:

[paste the specific phase instructions from Section 8]

Definition of Done for this phase (verify each item before considering it complete):

[paste the phase's Definition of Done checklist from Section 8]

Constraints:

- Follow the AI Agent Guardrails in Section 1 — in particular: no empty/blank containers,
 - no duplicated metric calculations, consistent CTA copy, no fabricated data.
- Mobile-first, dark theme default.
- All prices formatted ₹X,XX,XXX.XX (Indian format, en-IN locale), monospace font.
- Skeleton loaders while data is loading; EmptyState component when data is genuinely absent.
- Every component has a TypeScript props interface. No `any` types.
- React Query for all server data, Zustand for UI state only.

Output: complete, working TypeScript + React code for every file needed in this phase,

plus a short summary confirming each Definition of Done item is satisfied – flag anything that could not be fully completed rather than omitting it silently.

14. RECOMMENDED BUILD ORDER SUMMARY

Day	Phase	Focus
1-2	Phase 1	Bootstrap, routing, design system
3-4	Phase 2	Authentication
5-6	Phase 3	Market data layer + caching proxy
7	Phase 4	Dashboard
8-10	Phase 5	Advanced charting
11-13	Phase 6	Trading engine
14	Phase 7	Portfolio
15	Phase 8	Watchlist
16	Phase 9	News
17	Phase 10	Leaderboard
18	Phase 11	Profile & Settings
19	Phase 12	Premium / Payments
20	Phase 13	Help Center
21	Phase 14	PWA & Performance
22	Phase 15	Testing & Deployment

Total estimated build time: 22 focused development days with AI assistance.